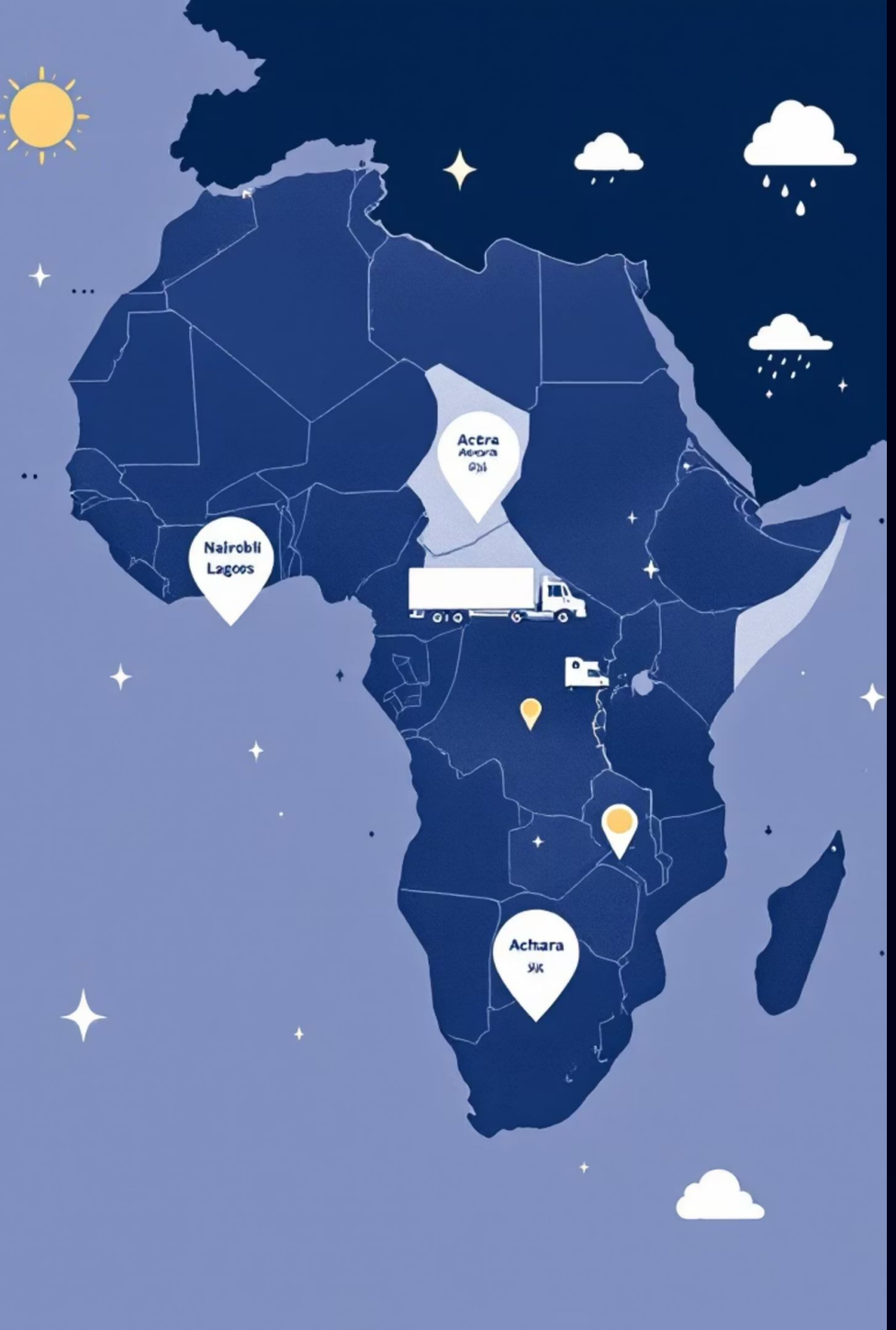
Building a Real-Time Weather Data Pipeline using MongoDB, Airbyte, and DuckDB for SkyLogix Transportation



Company Overview

SkyLogix Transportation is a logistics and delivery company headquartered in Nairobi, Kenya. With operations across Lagos, Accra, and Johannesburg, SkyLogix manages a fleet of over 1,200 delivery trucks. The company is focused on ensuring timely deliveries, minimizing risks, and optimizing routes in diverse African markets.

AMDARI®

AMDARI®

Current Business Challenge

SkyLogix faces increasing challenges in its day-to-day operations due to the unpredictable nature of weather across its service regions:

- Manual Data Gathering: Dispatch teams rely on third-party weather apps, which introduces delays and inconsistencies.
- Disparate Data Sources: No central repository exists for weather data, making trend analysis impossible.
- Operational Risks: Poor visibility into weather conditions has caused delivery delays, higher insurance costs, and safety issues.
- Inefficient Reporting: Lack of integration with logistics dashboards prevents proactive decision-making.

Rationale for the Project

Implementing a weather data pipeline will:

1. **Automate Data Collection**

   Continuously ingest weather data from APIs without manual checks.

2. **Improve Data Quality and Consistency**

   Use MongoDB as a staging database to standardize and store raw weather feeds.

3. **Enable Incremental Loading**

   Configure Airbyte to capture only new or updated weather data using a timestamp cursor.

4. **Enhance Decision-Making**

   Provide near real-time data in DuckDB for operational dashboards and analytics.

5. **Reduce Operational Costs**

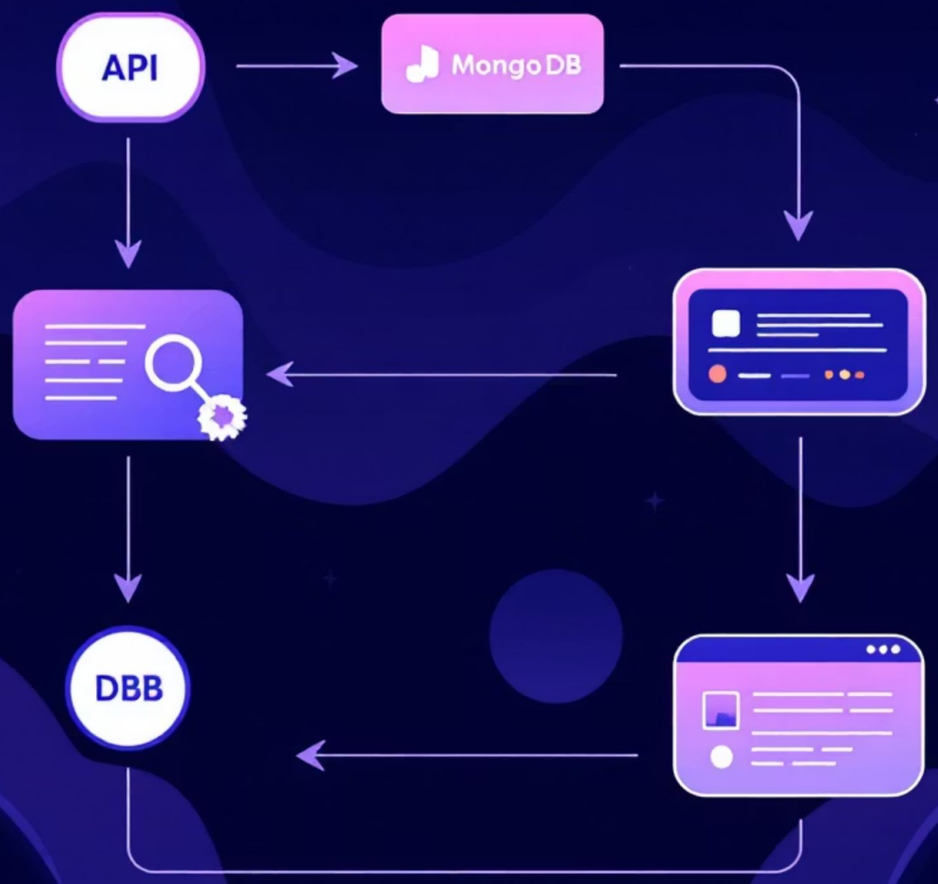
   Minimize weather-related delays and improve resource allocation.

AMDARI®

AMDARI®

Project Objectives

- Deploy MongoDB as the staging database for weather API feeds.
- Develop a Python-based ingestion script to poll weather APIs and upsert into MongoDB.
- Configure Airbyte (Docker) to replicate MongoDB data into DuckDB.
- Implement Incremental | Append + Deduped syncs using `updatedAt` as the cursor and `_id` as the primary key.
- Establish a clear data model to support analytics and dashboarding.



Project Scope: Deliverables

1. GitHub Repository

Documentation, Python ingestor script, and connection configuration files.

2. Automated Data Ingestion

Script to fetch weather API data for multiple cities and insert/update records in MongoDB.

3. Database Layer (MongoDB)

Collections for structured and raw weather data with appropriate indexing.

4. Data Pipeline Deployment (Airbyte)

Airbyte source: MongoDB. Airbyte destination: DuckDB warehouse file.

5. Data Model (Warehouse)

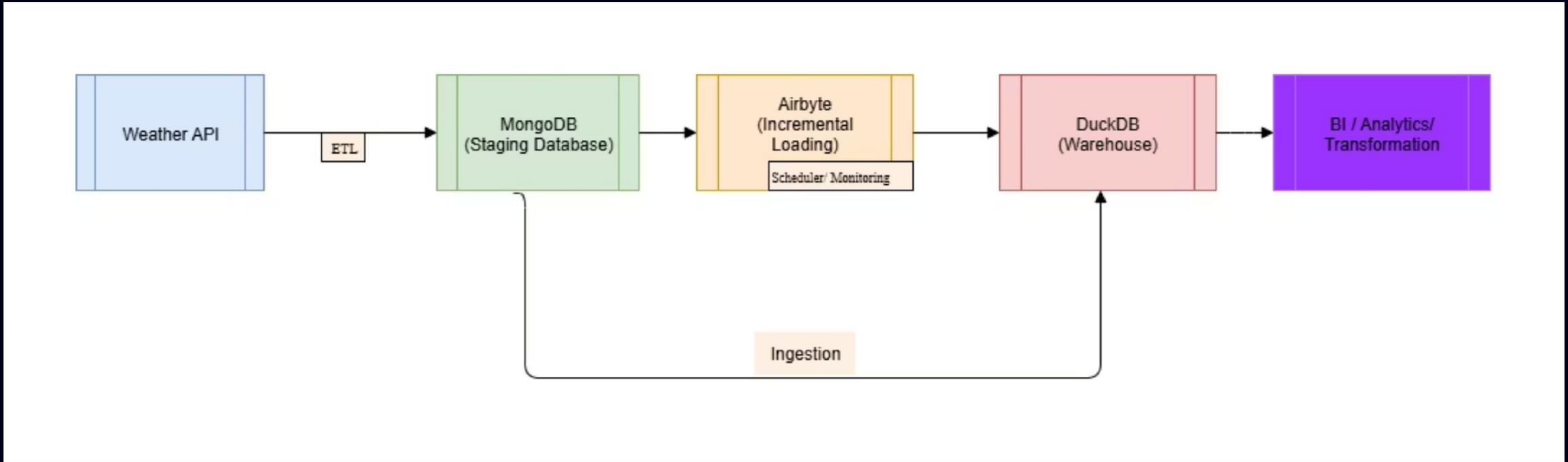
Clean tables in DuckDB for `weather_readings`, supporting incremental updates.

6. Analytics Enablement

Weather data accessible for dashboards and operational decision-making.

AMDARI®

Data Architecture



Tech Stack

Python

For building the API ingestor and upsert logic.

MongoDB

Staging database for raw and structured weather data.

Airbyte

Data integration and incremental replication.

DuckDB

Analytical warehouse for query-ready weather data.

GitHub

Version control for scripts and project documentation.

AMDARI®